

ETC5513 Mid-Semester Test Solutions

Michael Lydeamore

Multiple Choice

Which of these is not a structural component of a Quarto (qmd) document?

Which of these is **not** a structural component of a Quarto (qmd) document?

- YAML metadata
- Markdown text
- R code chunks
- Required PDF output format

Correct answer: Required PDF output format

Which statement about project folders is TRUE (best answer)?

Which statement about project folders is TRUE (best answer)?

- A project folder should contain all files needed to reproduce an analysis using only relative paths.
- A project folder must be an RStudio .Rproj file to be valid.
- A project folder should always use absolute file paths for portability.
- Project folders are only required when using git.

Correct answer: A project folder should contain all files needed to reproduce an analysis using only relative paths.

Which chunk option hides code but still shows results in the rendered document?

Which chunk option hides code but still shows results in the rendered document?

- `eval: false`
- `echo: false`
- `include: false`
- `message: false`

Correct answer: `echo: false`

For a table to be cross-referenceable in Quarto, its chunk label must start with:

For a table to be cross-referenceable in Quarto, its chunk label must start with:

- fig-
- tbl-
- tab-
- table-

Correct answer: `tbl-`

Which git command updates your local git database with remote changes but does NOT change your working tree?

Which git command updates your local git database with remote changes but does NOT change your working tree?

- `git pull`
- `git fetch`
- `git merge origin/main`
- `git clone`

Correct answer: `git fetch`

In the Positron Git pane, what does a green “U” icon beside a file mean?

In the Positron Git pane, what does a green “U” icon beside a file mean?

- The file is untracked and not yet staged for commit.
- The file has been updated on the remote repository.
- The file has been updated locally and staged for commit.
- The file is unchanged since the last commit.

Correct answer: The file is untracked and not yet staged for commit.

Which command sequence correctly stages the changes to your `analysis.R` file, makes a commit with a message, and pushes to origin main?

Which command sequence correctly stages the changes to your `analysis.R` file, makes a commit with a message, and pushes to origin main?

- `git commit -m "Updated model used for analysis" ; git add analysis.R ; git push origin main`
- `git add analysis.R ; git commit -m "Updated model used for analysis" ; git push origin main`
- `git push origin main ; git add Analysis.R ; git commit -m "Updated model used for analysis"`
- `git add analysis.R ; git push origin main ; git commit -m "Updated model used for analysis"`

Correct answer: `git add analysis.R ; git commit -m "Updated model used for analysis" ; git push origin main`

Which statement about git stash is correct?

Which statement about git stash is correct?

- `git stash` permanently stores changes on the remote server.
- `git stash apply` both applies the stash and removes it from the stash list.
- `git stash pop` applies the stash and removes it from the stash list.
- `git stash` must be used before every commit.

Correct answer: `git stash pop` applies the stash and removes it from the stash list.

Which of the following is the recommended rule for using git rebase in collaborative projects?

Which of the following is the recommended rule for using git rebase in collaborative projects?

- `Rebase main` on every feature branch publicly to keep history linear.
- Never rebase any branch locally.
- Avoid rebasing branches that have been published/shared with others.
- Always use rebase instead of merge for all merges.

Correct answer: Avoid rebasing branches that have been published/shared with others.

You need to undo a public (already pushed) commit while preserving history. Which command is the safest choice?

You need to undo a public (already pushed) commit while preserving history. Which command is the safest choice?

- `git reset --hard <commit>`
- `git revert <commit>`
- `git checkout <commit>`
- `git rebase -i <commit>`

Correct answer: `git revert <commit>`

Free-answer questions

Quarto YAML

Write a minimal YAML header for a Quarto document titled “Mid-semester Test”.

The document should:

- Render both a HTML and PDF output,
- Has yourself as the author,
- Has them HTML theme set to `cerulean`,
- Set global execute options to show all the code, and use the cache for all chunks, but does not print out warnings or errors.

Solution:

```

---
title: "Mid-semester Test"
author: "Your Name"
theme: cerulean
format:
  html: default
  pdf: default
execute:
  echo: true
  cache: true
  warning: false
  error: false
---

```

Figures from a code chunk

Write a single R Code chunk (including the header) that:

- Does not show the R code in the rendered output
- Sets the label to `fig-penguins` and the caption to “Penguin body mass by species”
- Produces a `ggplot` figure using the `penguins` dataset that shows a boxplot `body_mass_g` on the y-axis and `species` on the x-axis.

Then, write the markdown text to cross-reference this figure in the document.

You do not need to load any libraries or the dataset in your answer.

Solution:

```

#| echo: false
#| fig-cap: "Penguin body mass by species"
#| label: fig-penguins
ggplot(penguins, aes(x = species, y = body_mass_g)) +
  geom_boxplot()

```

.gitignore

Write a `.gitignore` snippet that ignores:

- R history files (`.Rhistory`)
- Workspace images (`.RData`)
- All csv files in the `data/` folder
- All files in the `output/` folder except for `output/important_results.csv`

Solution:

```
.Rhistory
.RData
data/*.csv
output/*
!output/important_results.csv
```

The states of git

Describe the three main states of files in git, and how they relate to the workflow of making changes to your files. You can include git commands to help in your explanations.

Explain how you ensure that your changes are synced to the the remote repository on GitHub.

Solution:

In git, files can be in three main states: modified, staged, and committed. When you first make changes to a file, it is in the modified state. You can see which files are modified using `git status`. To include these changes in the next commit, you need to stage them using `git add <file>`. Once staged, the file is in the staged state. Finally, when you run `git commit`, the staged changes are saved as a new commit in your local repository.

To sync your changes to the remote repository on GitHub, you need to push your commits using a combination of `git push origin <branch>` and `git pull origin <branch>`. This will upload your local commits to the corresponding branch on GitHub, making them available to others and ensuring that your work is backed up remotely.

Safely undoing a pushed commit

You have pushed a commit with hash `abc1234` to `origin/main` and need to undo it's changes but preserve history. Write the git command(s) you would run to safely undo that commit on main. You can assume you are currently on the main branch and your working tree is clean and up to date. Briefly (one short line) state what the command does.

Solution:

```
git revert abc1234
```

This command creates a new commit that is the reverse of the changes introduced by commit `abc1234`, while preserving the commit history.